CSAI 202

Introduction to Database Systems



Lec 03

Data Modeling using the ER Model



Outline

- Overview of Database Design Process
- Example Database Application (COMPANY)
- ER Model Concepts
- ER Diagrams - Notation
- ER Diagram for COMPANY Schema
- Relationships of Higher Degree

Database Design Process

- Two main activities:
  - Database design
  - Applications design
- Focus in this chapter on database design
  - To design the conceptual schema for a database application
- Applications design focuses on the programs and interfaces that access the database
  - Generally considered part of software engineering

Database Design Process

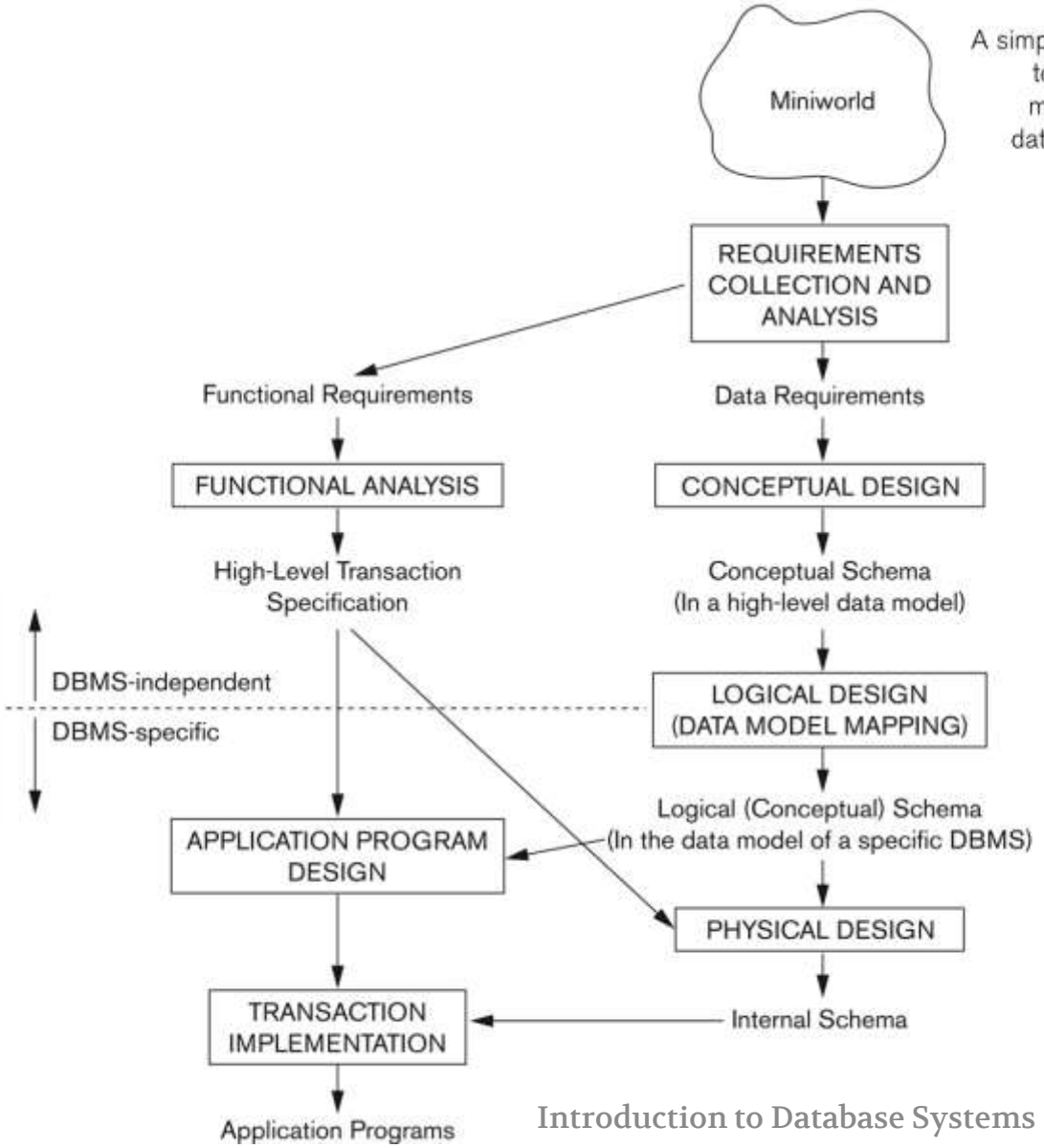


Figure 3.1
A simplified diagram to illustrate the main phases of database design.

ER Model Concepts

- Entities and Attributes
  - Entities are ***specific objects*** or things in the mini-world that are represented in the database.
    - For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT
  - Attributes are ***properties*** used to describe an entity.
    - For example an EMPLOYEE entity may have the attributes Name, SSN, Address, Sex, BirthDate
  - A specific entity will have a value for each of its attributes.
    - For example a specific employee entity may have Name='John Smith', SSN='123456789', Address ='731, Fondren, Houston, TX', Sex='M', BirthDate='09-JAN-55‘
  - Each attribute has a value set (or data type) associated with it – e.g. integer, string, subrange, enumerated type, …

Types of Attributes (1)

- Simple
  - Each entity has a single atomic value for the attribute. For example, SSN or Gender.
- Composite
  - The attribute may be composed of several components. For example:
    - Address(Apt#, House#, Street, City, State, ZipCode, Country), or
    - Name(FirstName, MiddleName, LastName).
    - Composition may form a hierarchy where some components are themselves composite.

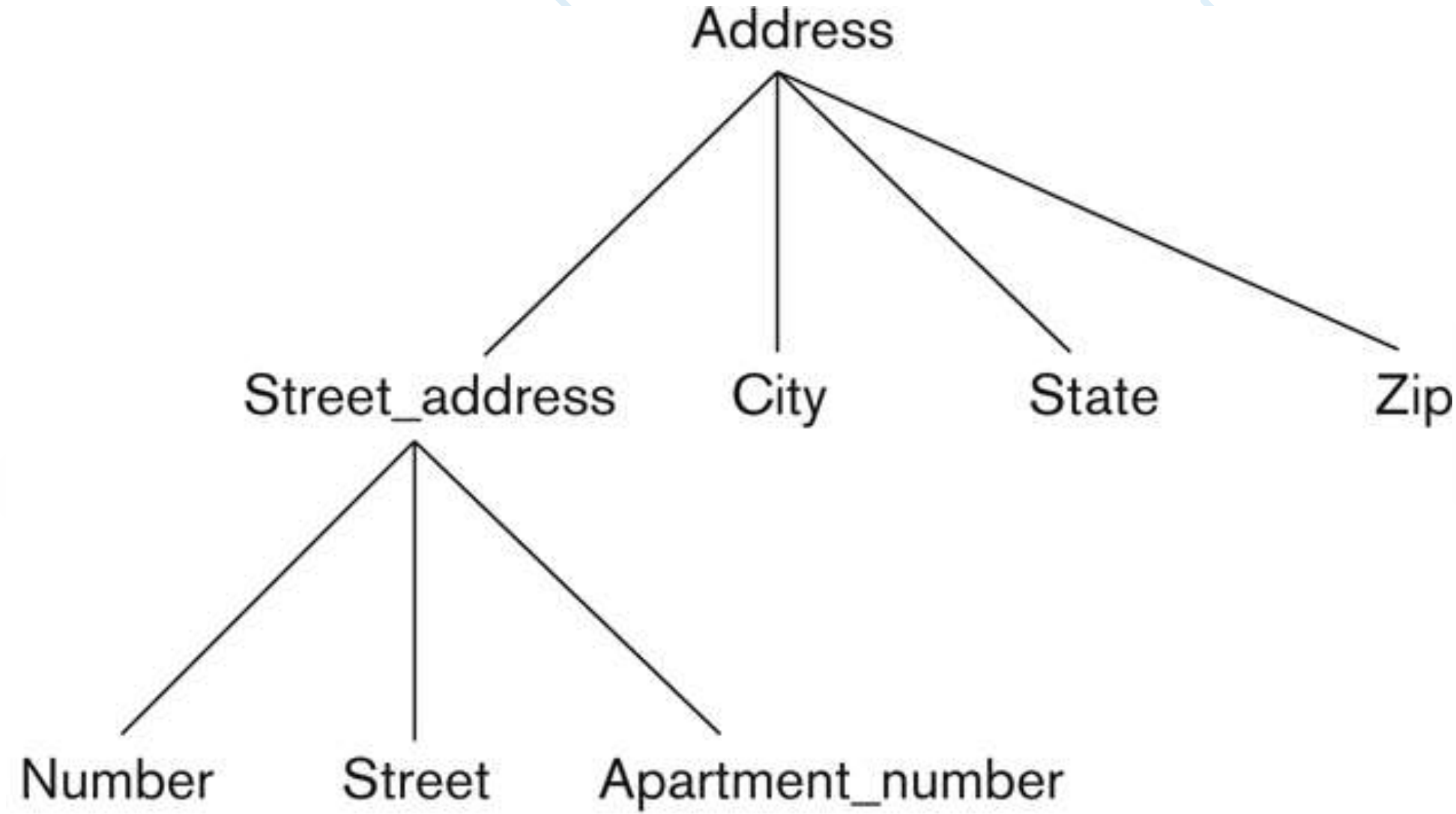
Types of Attributes (2)

- Multi-valued
  - An entity may have multiple values for that attribute.
    - For example, Color of a CAR or PreviousDegrees of a STUDENT.
  - Denoted as {Color} or {PreviousDegrees}.
- Derived attributes
  - Some attribute values can be derived from related entities, for example:
    - An attribute Number_Of_Employees of a department entity can be derived by counting number of employees related to that department

Types of Attributes (3)

- In general, composite and multi-valued attributes may be nested arbitrarily to any number of levels, although this is rare.
  - For example, ***PreviousDegrees*** of a STUDENT is a composite multi-valued attribute denoted by **{PreviousDegrees (College, Year, Degree, Field)}**
  - Multiple PreviousDegrees values can exist, each has four subcomponent attributes:
    - College, Year, Degree, Field

Composite attribute



Entity Types & Key Attributes (1)

- Entities with the same basic attributes are grouped or typed into an entity type.
  - ★For example, the entity type EMPLOYEE and PROJECT.
- An ***attribute*** of an entity type for which each entity must have a ***unique value*** is called a **key attribute** of the entity type.
  - ★For example, SSN of EMPLOYEE.

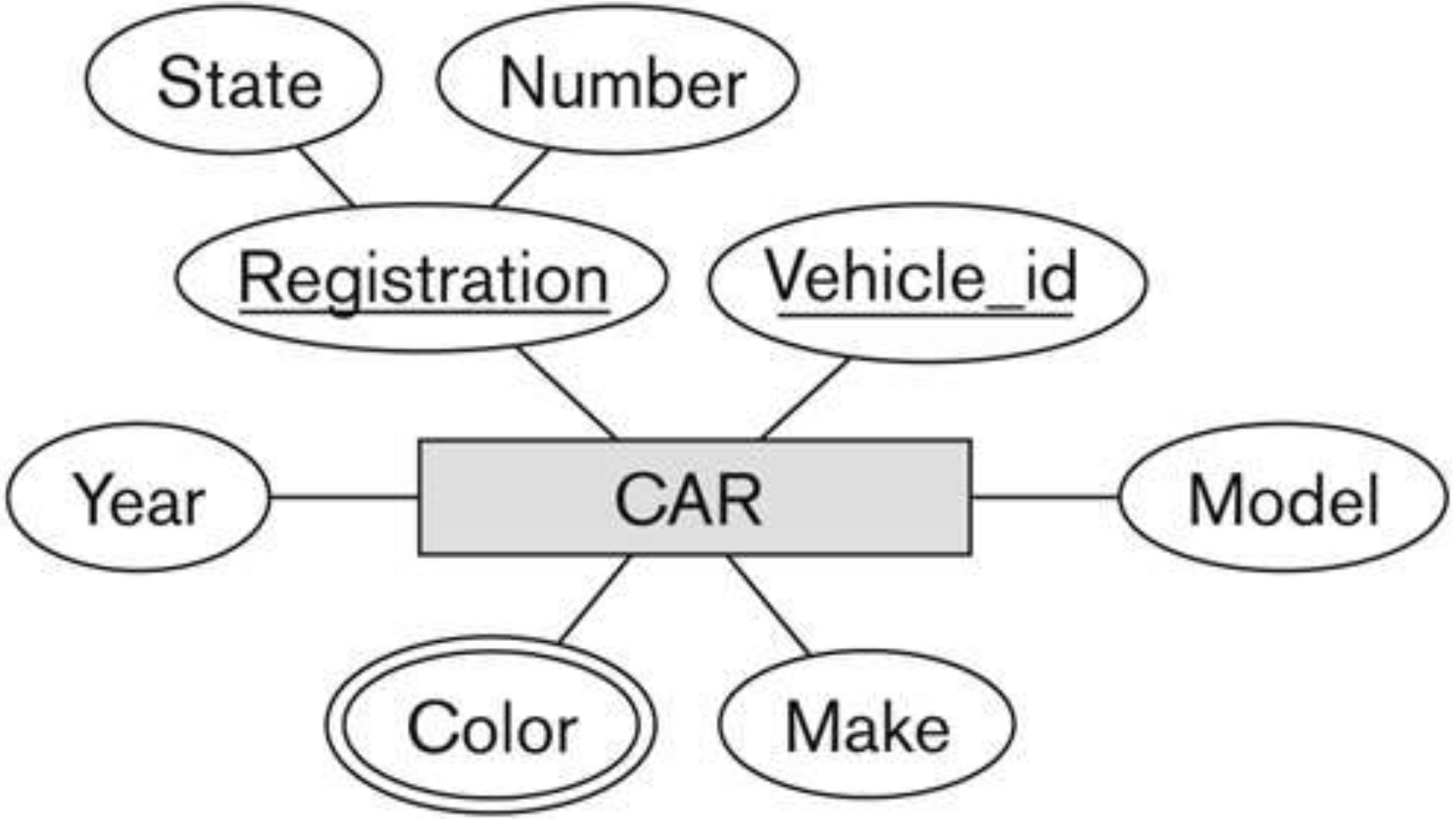
Entity Types & Key Attributes (2)

- A key attribute may be composite.
  - ★ *VehicleTagNumber* is a key of the CAR entity type with components (*Number, State*).
- An entity type may have more than one key.
  - ★ The CAR entity type may have two keys:
    - VehicleIdentificationNumber (popularly called VIN)
    - VehicleTagNumber (Number, State), aka license plate number.
- Each key is **underlined**

Displaying an Entity type

- In ER diagrams, an entity type is displayed in a **rectangular box**
- Attributes are displayed in **ovals**
  - Each attribute is connected to its entity type
  - Components of a composite attribute are connected to the oval representing the composite attribute
  - Each key attribute is ***underlined***
  - Multivalued attributes displayed in ***double ovals***
- See CAR example on next slide

Entity Type CAR with two keys



Entity Set

- Each entity type will have a collection of entities stored in the database
  - ★ Called the ***entity set***
- Previous slide shows three CAR entity instances in the entity set for CAR
- Same name (CAR) used to refer to both the entity type and the entity set
- Entity set is the current state of the entities of that type that are stored in the database

CAR corresponding Entity Set

CAR

Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR_1

((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR_2

((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR_3

((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

Example COMPANY Database

- We need to create a database schema design based on the following (simplified) *requirements* of the COMPANY Database:
  - The company is organized into DEPARTMENTs.
  - Each **department** has a *name, number* and an *employee* who *manages* the department.
  - We keep track of the *start date* of the department manager.
  - A department may have several *locations*.

  - Each department *controls* a number of PROJECTs.
  - Each **project** has a *unique name, unique number* and is located at a single *location*.

COMPANY Database (Contd.)

- We store each EMPLOYEE's social security number, address, salary, sex, and birthdate.
  - Each employee *works for* one department but may *work on* several projects.
  - We keep track of the *number of hours per week* that an employee currently works on each project.
  - We also keep track of the *direct supervisor* of each employee.

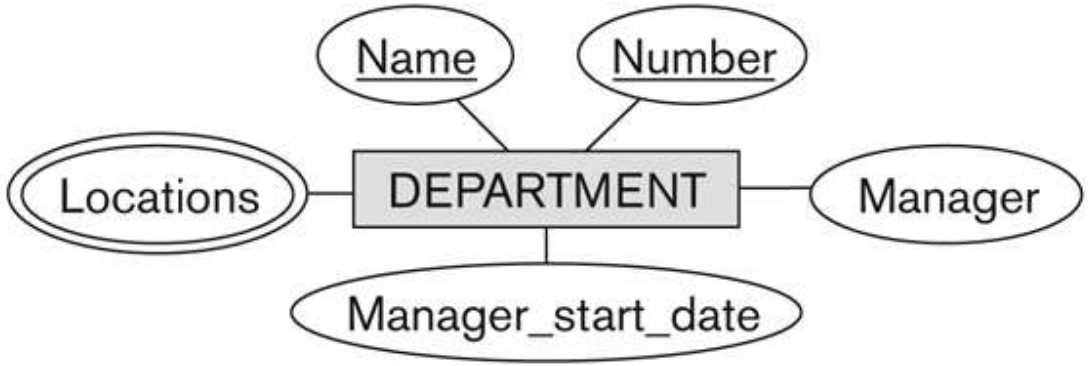
- Each employee may *have* a number of DEPENDENTs.
  - For each **dependent**, we keep track of their *name, sex, birthdate*, and *relationship* to the employee.

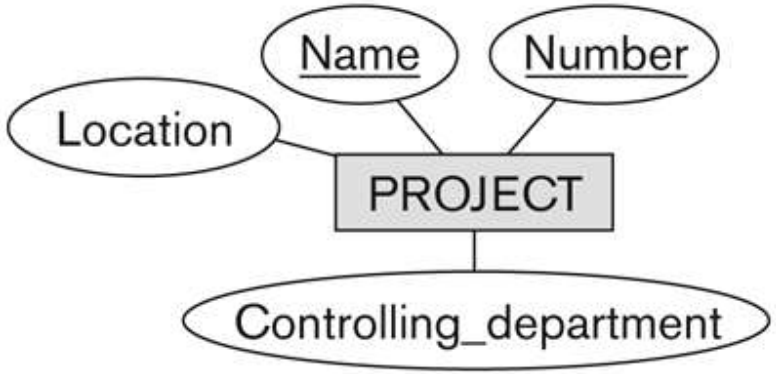
Initial Design of Entity Types for the COMPANY Database Schema

- Based on the requirements, we can identify four initial entity types in the COMPANY database:
  - ★DEPARTMENT
  - ★PROJECT
  - ★EMPLOYEE
  - ★DEPENDENT
- Their initial design is shown on the following slide
- The initial attributes shown are derived from the requirements description

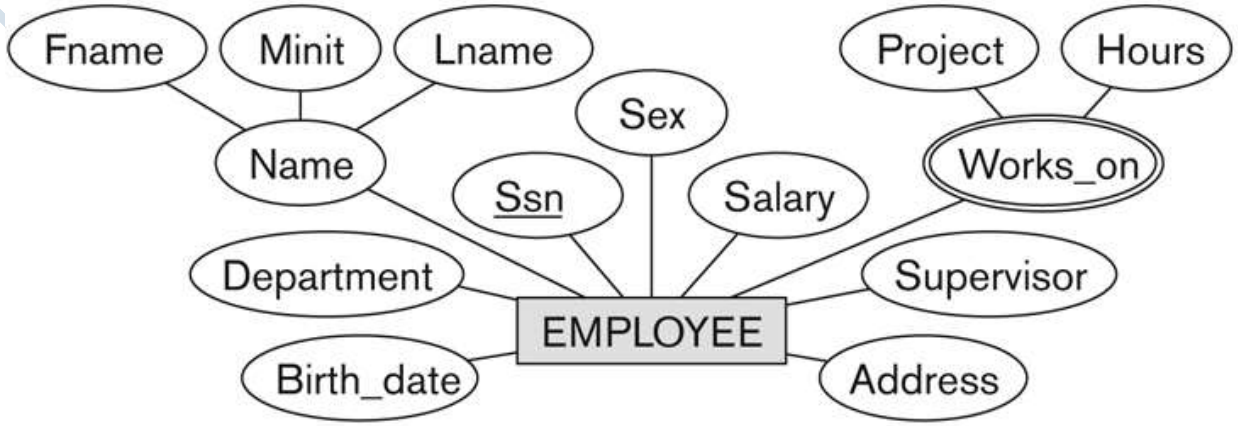
Initial Design of Entity Types

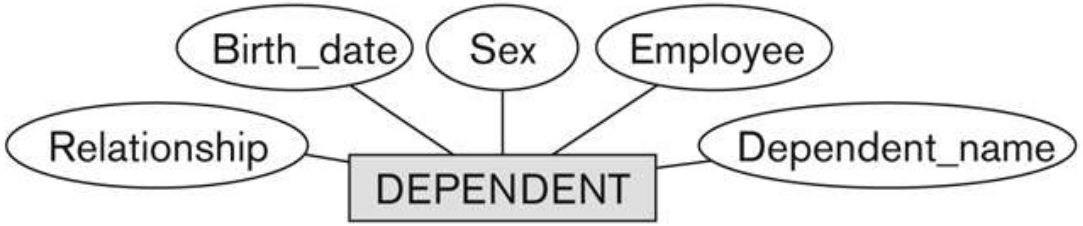
DEPARTMENT, PROJECT





EMPLOYEE, DEPENDENT





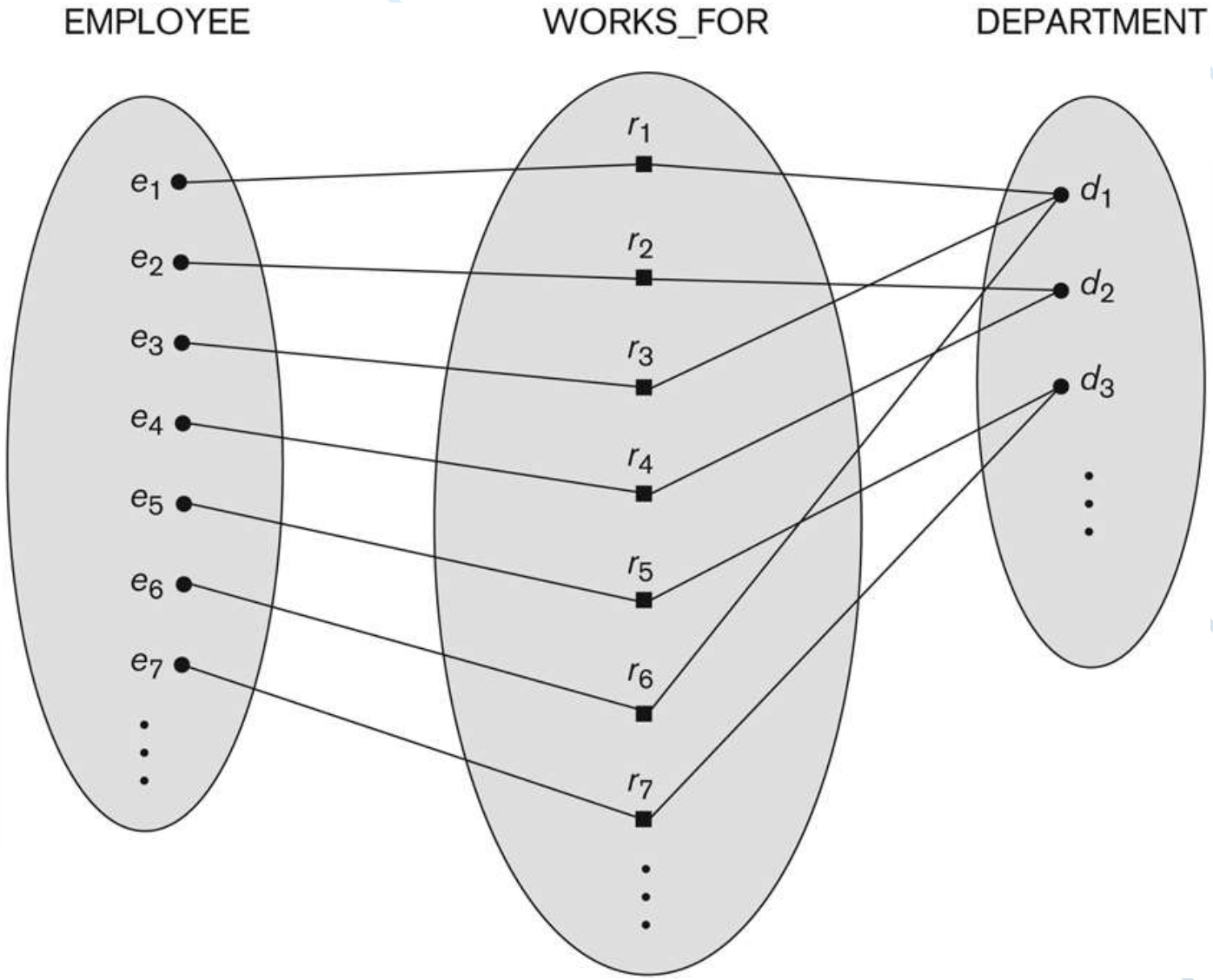
Refining the initial design

- **Introducing relationships**
- The initial design is typically not complete
- Some aspects in the requirements will be represented as relationships
- ER model has three main concepts:
  - Entities (and their entity types and entity sets)
  - Attributes (simple, composite, multivalued)
  - Relationships (and their relationship types and relationship sets)

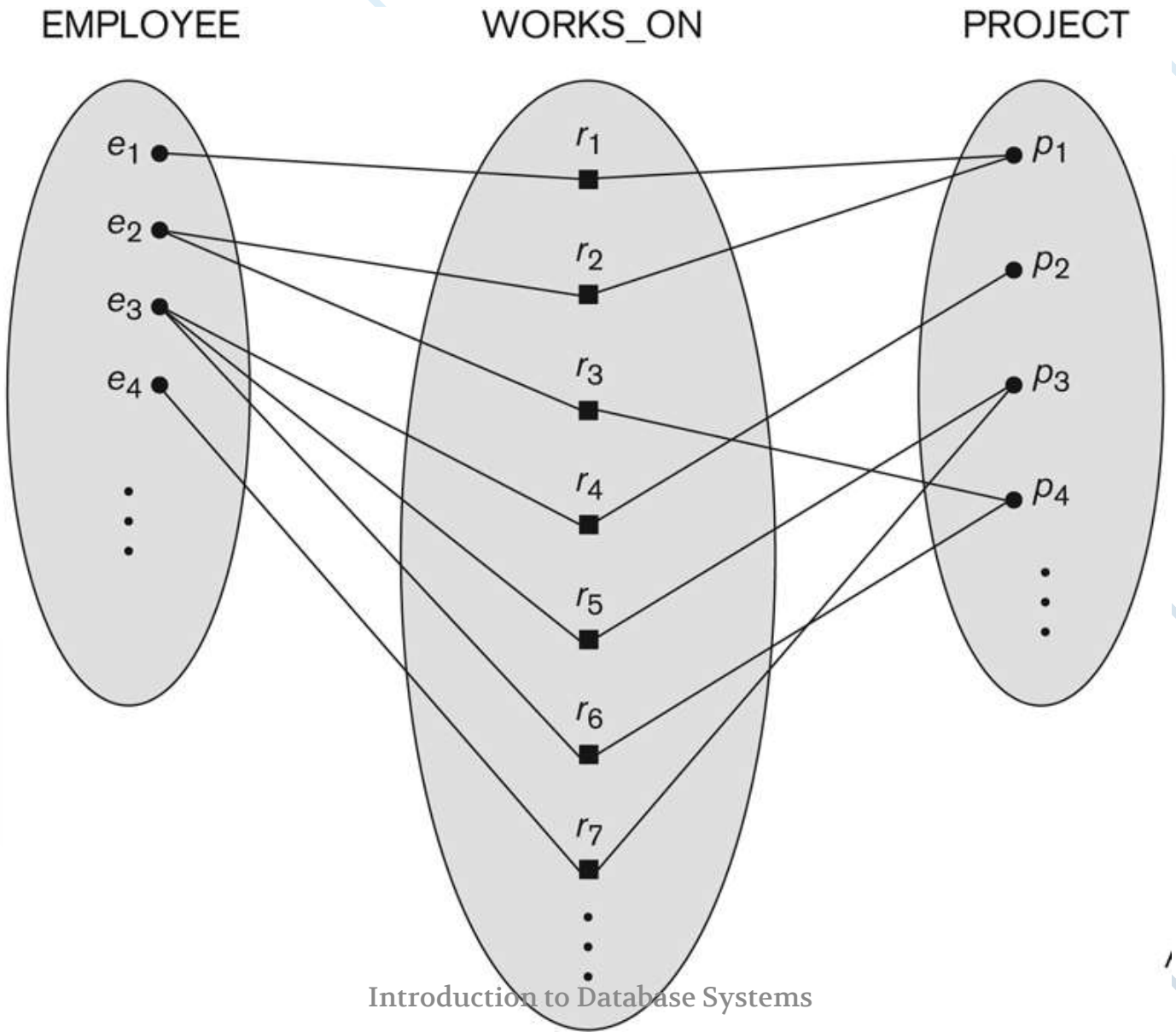
Relationships & Relationship Types (1)

- A **relationship** relates two or more distinct entities with a specific meaning.
  - For example,
    - EMPLOYEE John Smith *works on* the ProductX PROJECT
    - EMPLOYEE Franklin Wong *manages* the Research DEPARTMENT.
- Relationships of the same type are grouped or typed into a **relationship type**.
  - For example,
    - *WORKS_ON* relationship type in which EMPLOYEEs and PROJECTs participate
    - *MANAGES* relationship type in which EMPLOYEEs and DEPARTMENTs participate.
- The *degree of a relationship type* is the *number of participating entity types*.
  - Both *MANAGES* and *WORKS_ON* are **binary** relationships.

Relationship instances of the N:1



Relationship instances of the M:N



Type vs. Set

Relationship Type

- Is the schema description of a relationship
- Identifies the relationship name and the participating entity types
- Also identifies certain relationship constraints

Relationship Set

- The current set of relationship instances represented in the database
- The current state of a relationship type

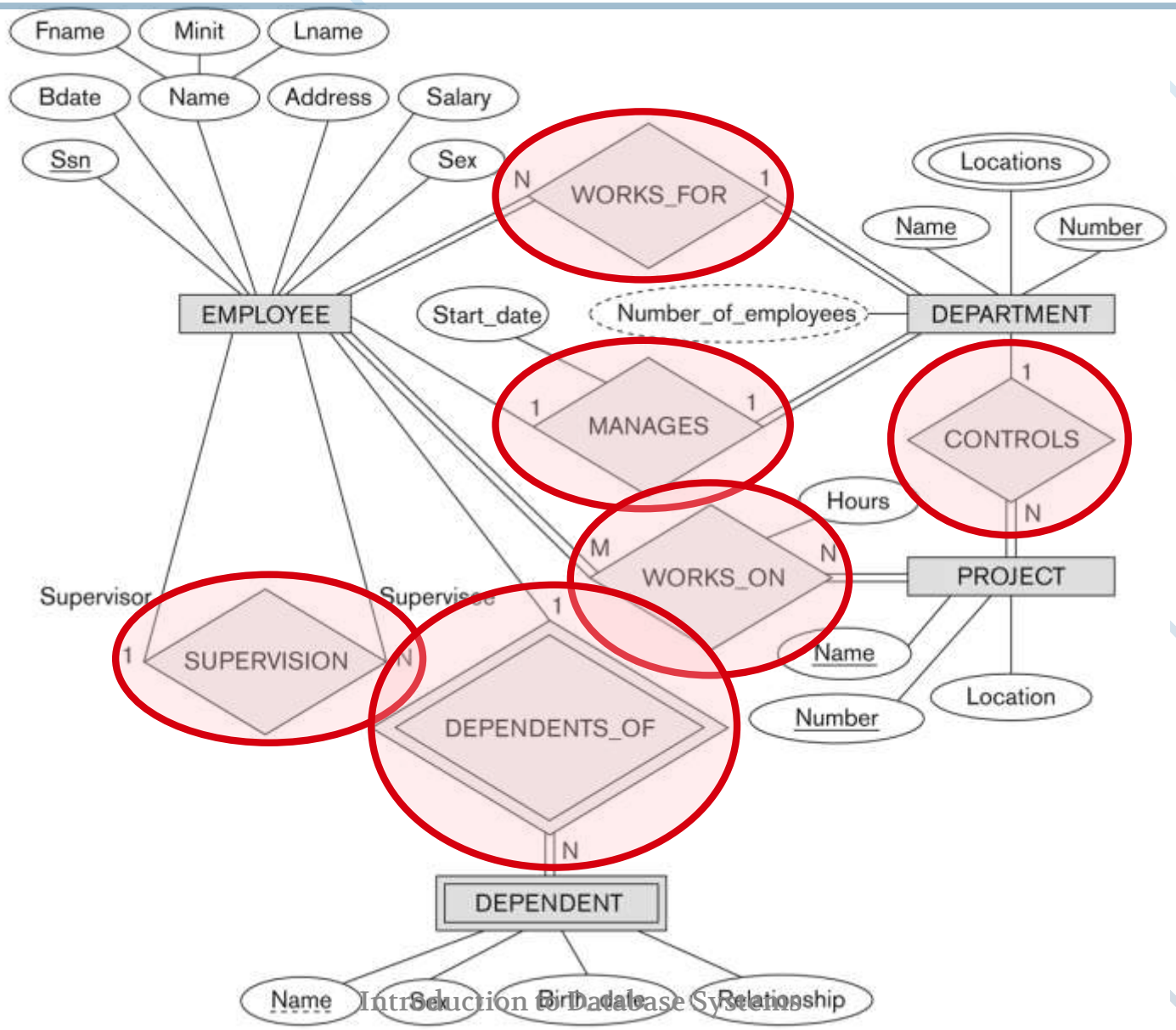
Type vs. Set

- Previous figures displayed the relationship sets
- Each instance in the set relates individual participating entities – one from each participating entity type
- In ER diagrams, we represent the *relationship type* as follows:
  - ★ ***Diamond-shaped box*** is used to display a relationship type
  - ★ Connected to the participating entity types via straight lines

Refining the COMPANY database schema

- **Introducing relationships**
- By examining the requirements, six relationship types are identified
- All are binary relationships(degree 2)
- Listed below with their participating entity types:
  - ★*WORKS_FOR* (between EMPLOYEE, DEPARTMENT)
  - ★*MANAGES* (also between EMPLOYEE, DEPARTMENT)
  - ★*CONTROLS* (between DEPARTMENT, PROJECT)
  - ★*WORKS_ON* (between EMPLOYEE, PROJECT)
  - ★*SUPERVISION* (between EMPLOYEE (as subordinate), EMPLOYEE (as supervisor))
  - ★*DEPENDENTS_OF* (between EMPLOYEE, DEPENDENT)

Relationship Types



Discussion on Relationship Types

- In the refined design, some attributes from the initial entity types are refined into relationships:
  - **Manager of DEPARTMENT → MANAGES**
  - **Works_on of EMPLOYEE → WORKS_ON**
  - **Department of EMPLOYEE → WORKS_FOR**
  - etc
- In general, more than one relationship type can exist between the same participating entity types
  - MANAGES and WORKS_FOR are distinct relationship types between EMPLOYEE and DEPARTMENT
  - Different meanings and different relationship instances.

Recursive Relationship Type

- A relationship type with the same participating entity type in distinct roles
- Example: the **SUPERVISION** relationship
- EMPLOYEE participates twice in two distinct roles:
  - supervisor (or boss) role
  - supervisee (or subordinate) role
- Each relationship instance relates two distinct EMPLOYEE entities:
  - One employee in *supervisor* role
  - One employee in *supervisee* role

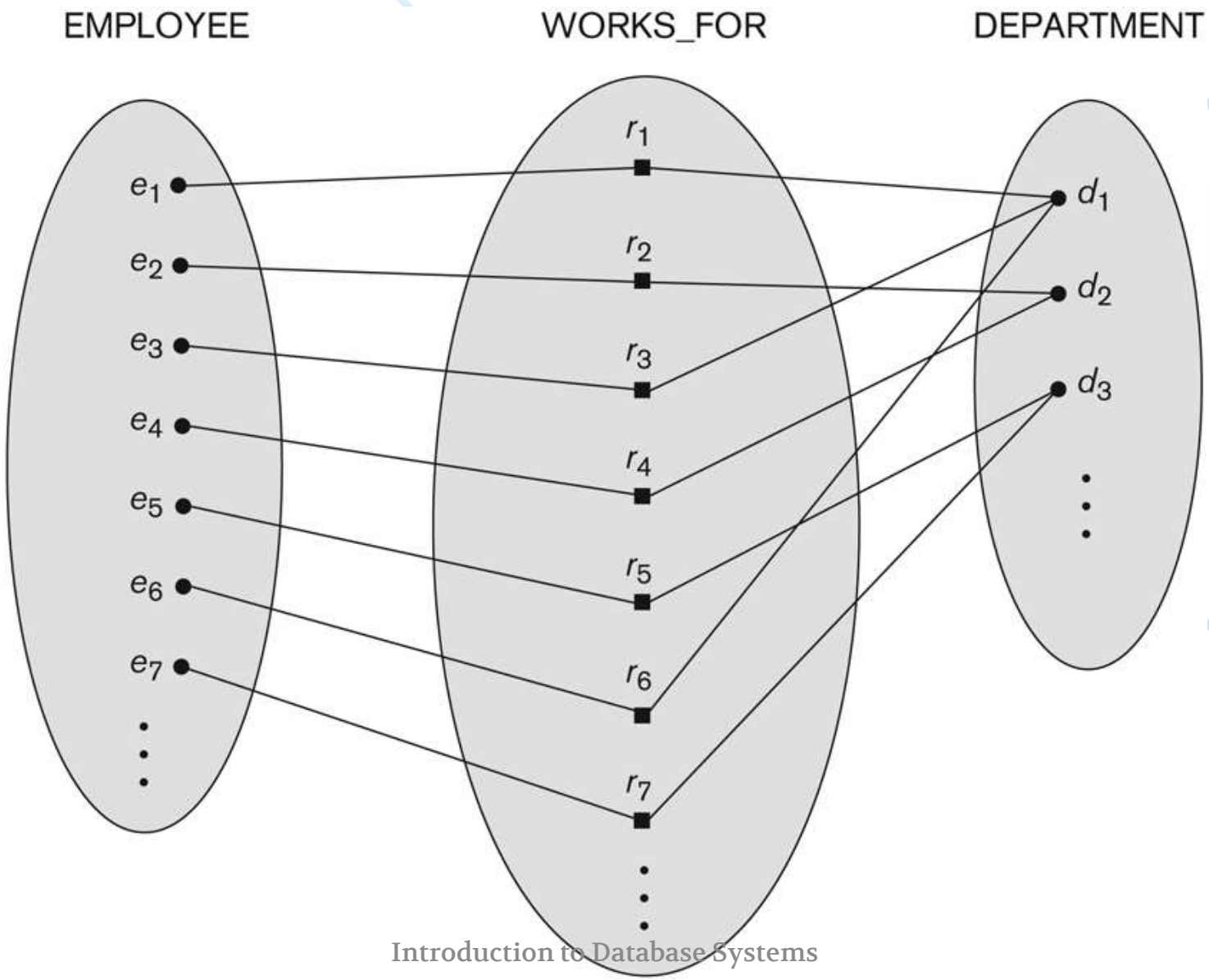
Weak Entity Types

- An entity that **does not have a key attribute**
- A weak entity *must participate in an identifying relationship type with an owner or identifying entity type*
- Entities are identified by the combination of:
  - A partial key of the weak entity type
  - The particular entity they are related to in the identifying entity type
- Example:
  - A DEPENDENT entity is identified by the dependent's first name, and the specific EMPLOYEE with whom the dependent is related
  - Name of DEPENDENT is the partial key
  - DEPENDENT is a weak entity type
  - EMPLOYEE is its identifying entity type via the identifying relationship type DEPENDENT_OF

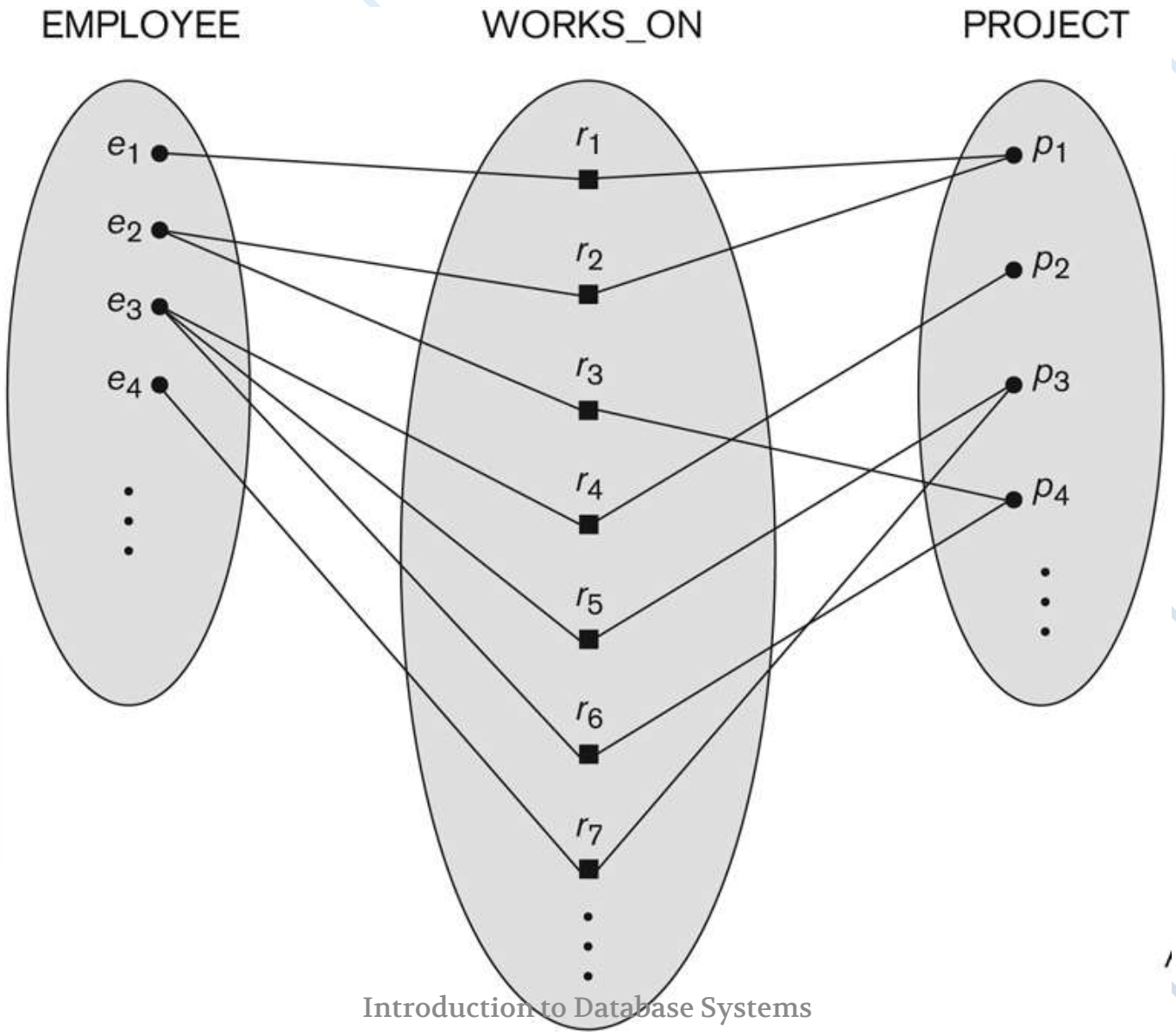
Constraints on Relationships

- Constraints on Relationship Types
  - ★(Also known as ratio constraints)
  - ★**Cardinality Ratio** (specifies ***maximum*** participation)
    - One-to-one (1:1)
    - One-to-many (1:N) or Many-to-one (N:1)
    - Many-to-many (M:N)
  - ★**Existence Dependency Constraint** (specifies ***minimum*** participation) (also called **participation constraint**)
    - zero (optional participation, not existence-dependent)
    - one or more (mandatory participation, existence-dependent)

Many-to-one (N:1) Relationship



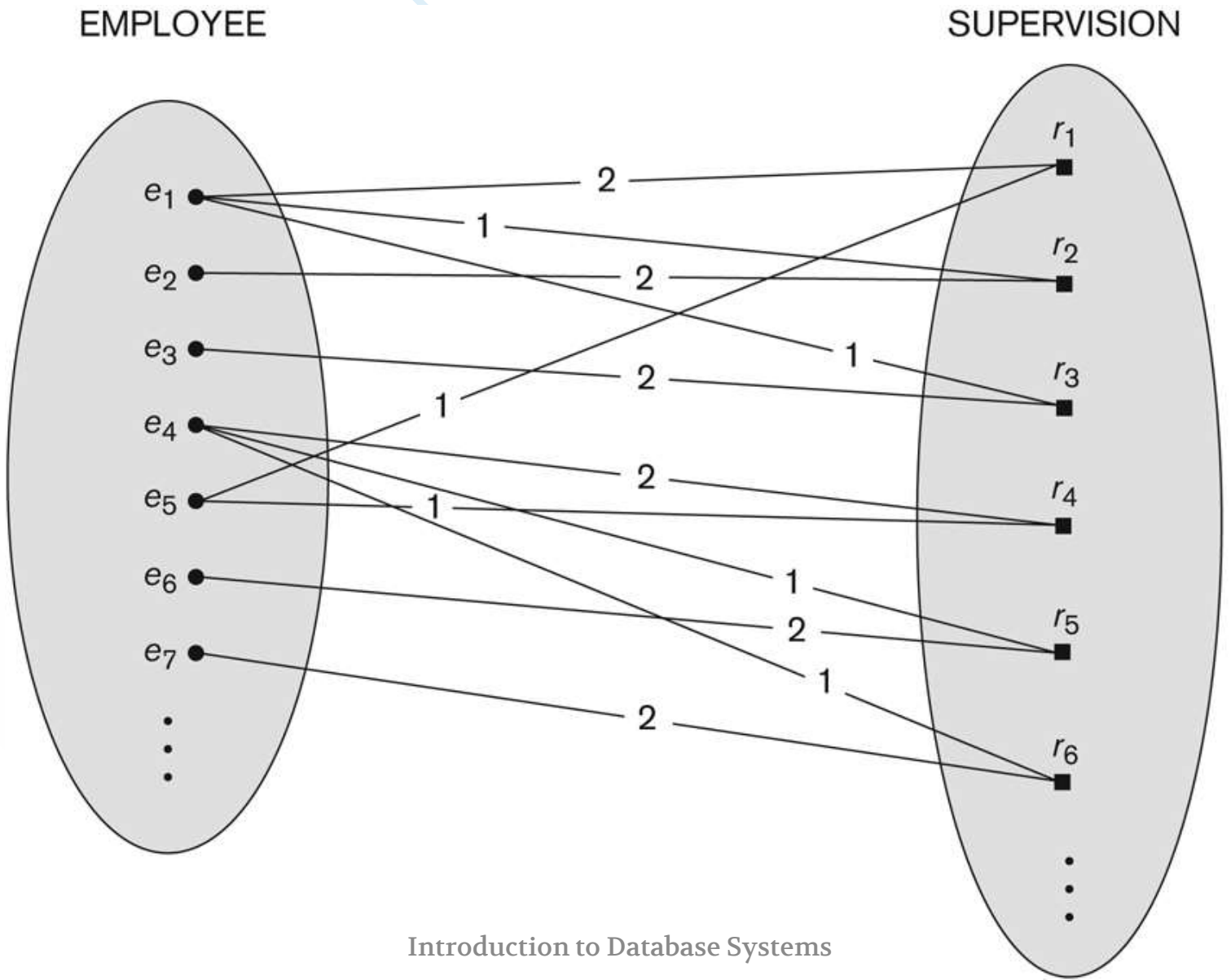
Many-to-many (M:N) Relationship



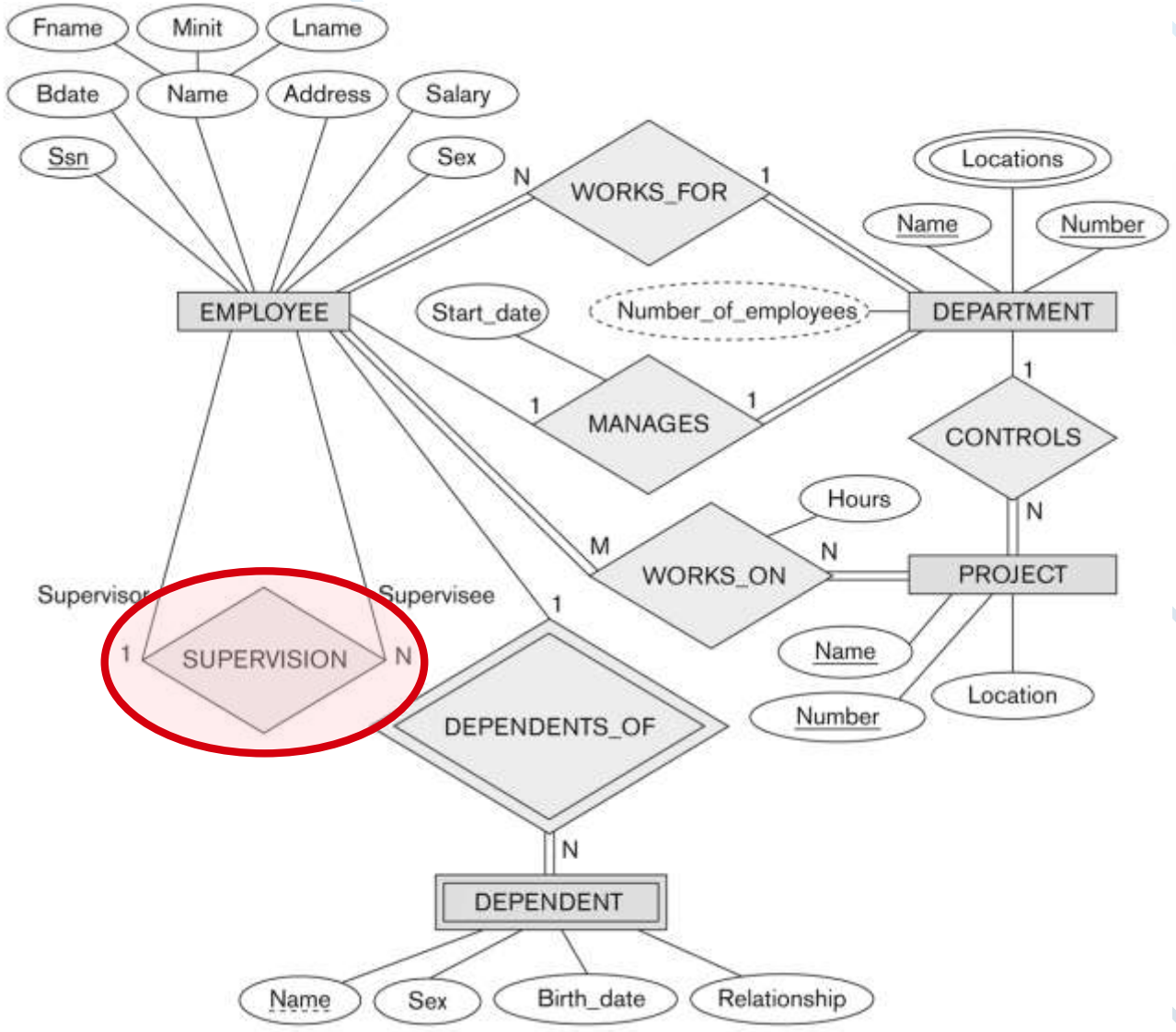
Displaying a recursive relationship

- In a recursive relationship type.
  - ★ Both participations are same entity type in different roles.
  - ★ For example, SUPERVISION relationships between EMPLOYEE (in role of supervisor or boss) and (another) EMPLOYEE (in role of subordinate or worker).
- In following figure, first role participation labeled with 1 and second role participation labeled with 2.
- In ER diagram, need to display role names to distinguish participations.

A Recursive Relationship Supervision



Recursive Relationship



Attributes of Relationship types

- A relationship type can have attributes:
  - For example, HoursPerWeek of WORKS_ON
  - Its value for each relationship instance describes the number of hours per week that an EMPLOYEE works on a PROJECT.
    - A value of HoursPerWeek depends on a particular (employee, project) combination
  - Most relationship attributes are used with M:N relationships
    - In 1:N relationships, they can be transferred to the entity type on the N-side of the relationship

Attribute of a Relationship



Notation for Constraints on Relationships

- Cardinality ratio (of a binary relationship): 1:1, 1:N, N:1, or M:N
  - ★Shown by placing appropriate numbers on the relationship edges.
- Participation constraint (on each participating entity type): total (called existence dependency) or partial.
  - ★Total shown by double line, partial by single line.
- NOTE: These are easy to specify for Binary Relationship Types.

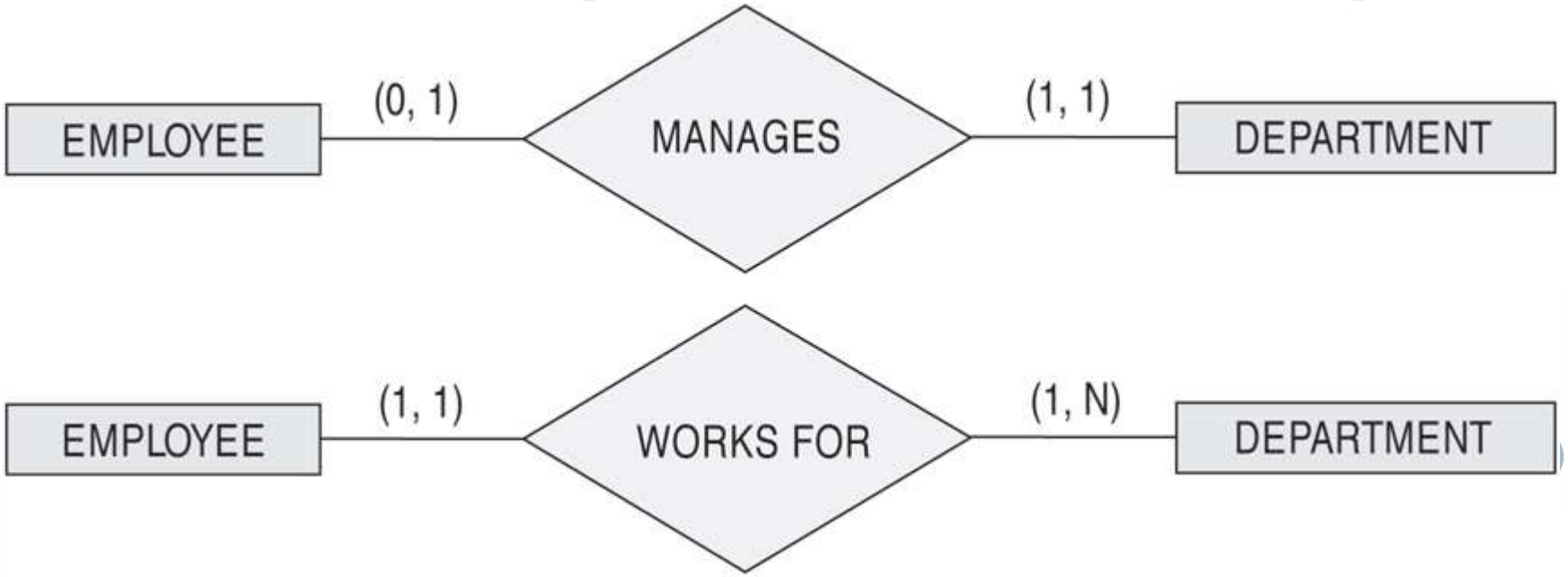
(min, max) notation

- Alternative (min, max) notation for relationship structural constraints:
- Specified on each participation of an entity type E in a relationship type R
- Specifies that each entity e in E participates in at least min and at most max relationship instances in R
- Default(no constraint): min=0, max=n (signifying no limit)
- Must have min≤max, min ≥ 0, max≥1
- Derived from the knowledge of mini-world constraints

Examples (min, max) notation

- A department has exactly one manager and an employee can manage at most one department.
  - Specify (0,1) for participation of EMPLOYEE in MANAGES
  - Specify (1,1) for participation of DEPARTMENT in MANAGES
- An employee can work for exactly one department but a department can have any number of employees.
  - Specify (1,1) for participation of EMPLOYEE in WORKS_FOR
  - Specify (1,n) for participation of DEPARTMENT in WORKS_FOR

The (min,max) notation



- Read the min, max numbers next to the entity type and looking away from the entity type

COMPANY ER using (min, max)

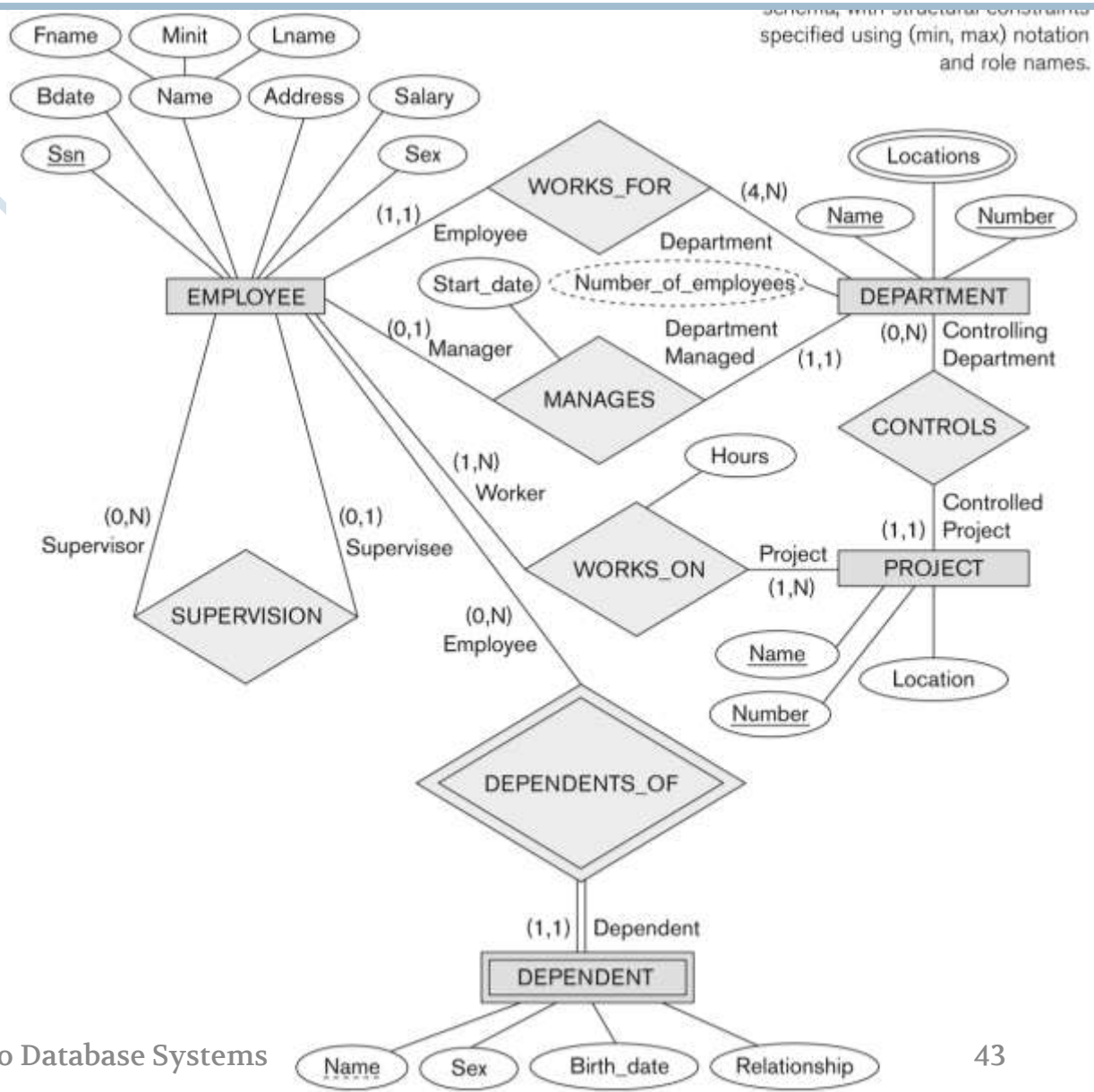
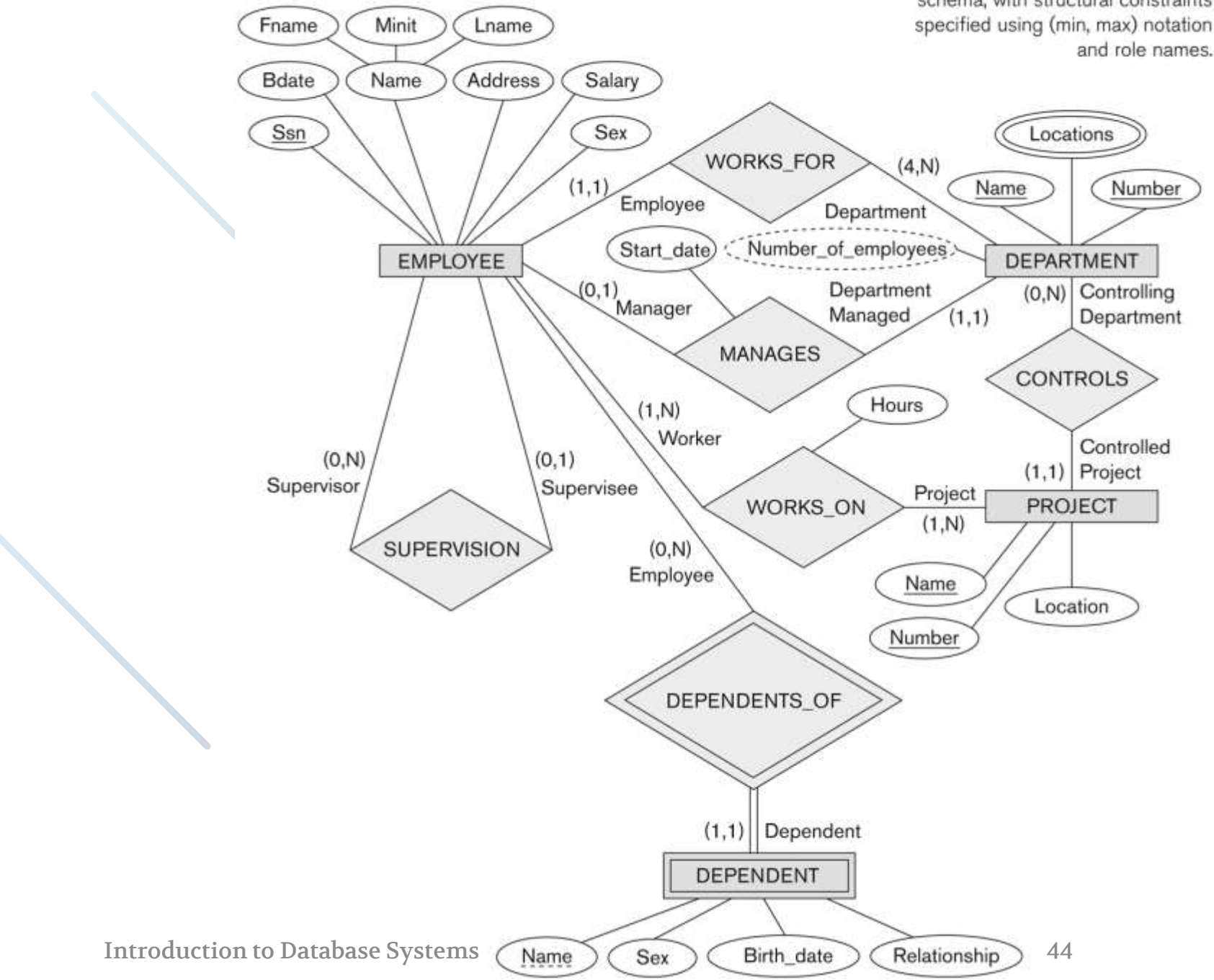
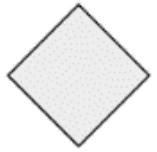
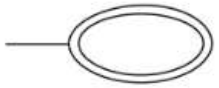
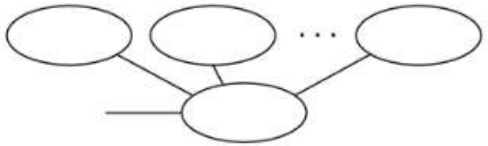
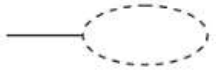
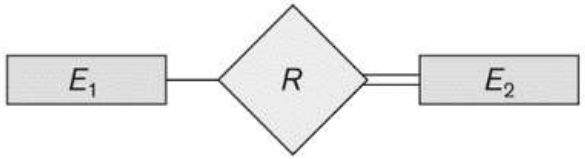
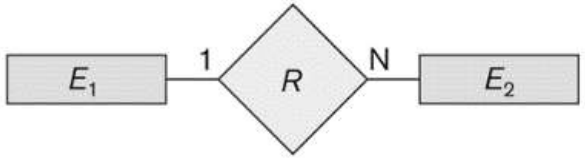
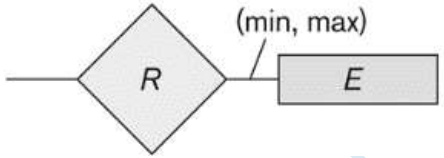


Figure 3.15
ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.



Summary of notation for ER diagrams

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Indentifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1: N for E_1:E_2 in R
	Structural Constraint (min, max) on Participation of E in R

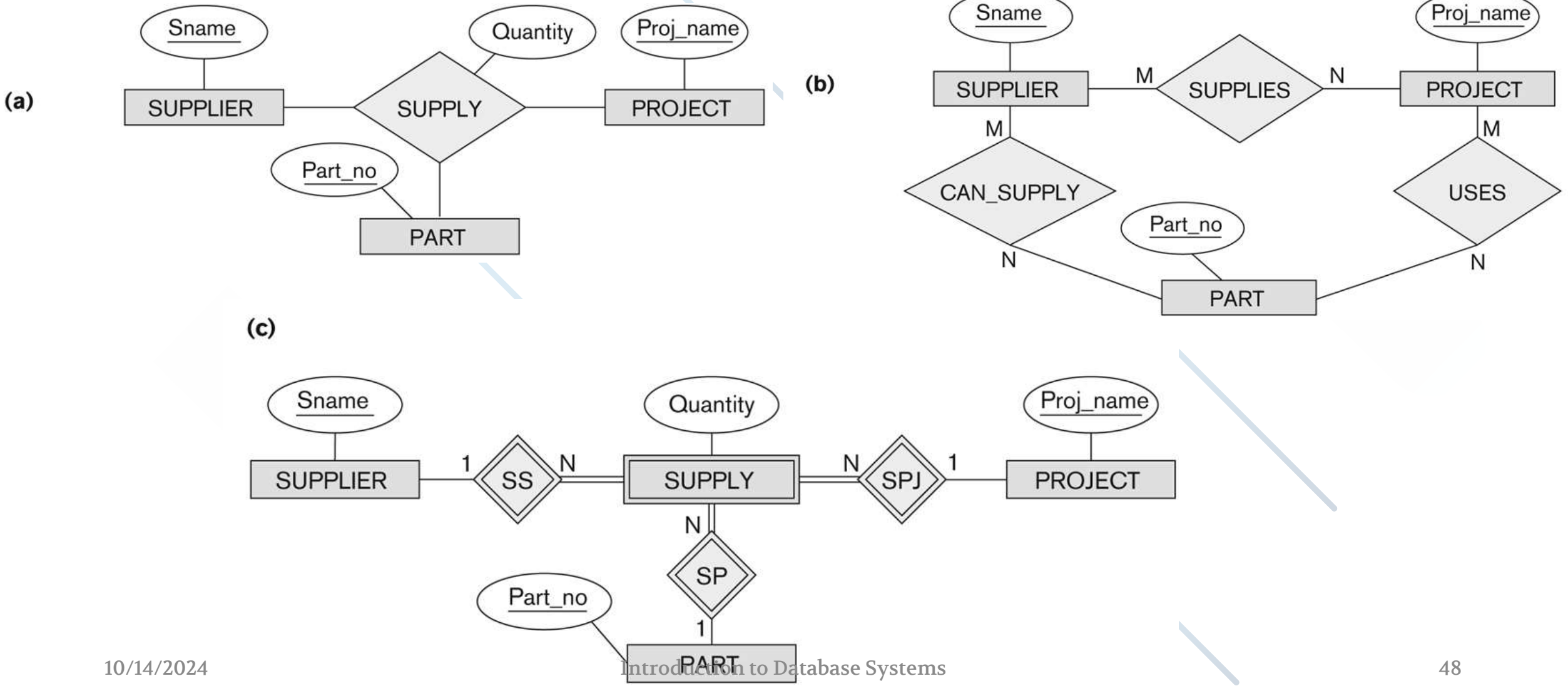
Relationships of Higher Degree

- Relationship types of *degree 2* are called *binary*
- Relationship types of *degree 3* are called *ternary* and of *degree n* are called *n-ary*
- In general, *an n-ary relationship is not equivalent to n binary relationships*
- Constraints are harder to specify for higher-degree relationships (n > 2) than for binary relationships

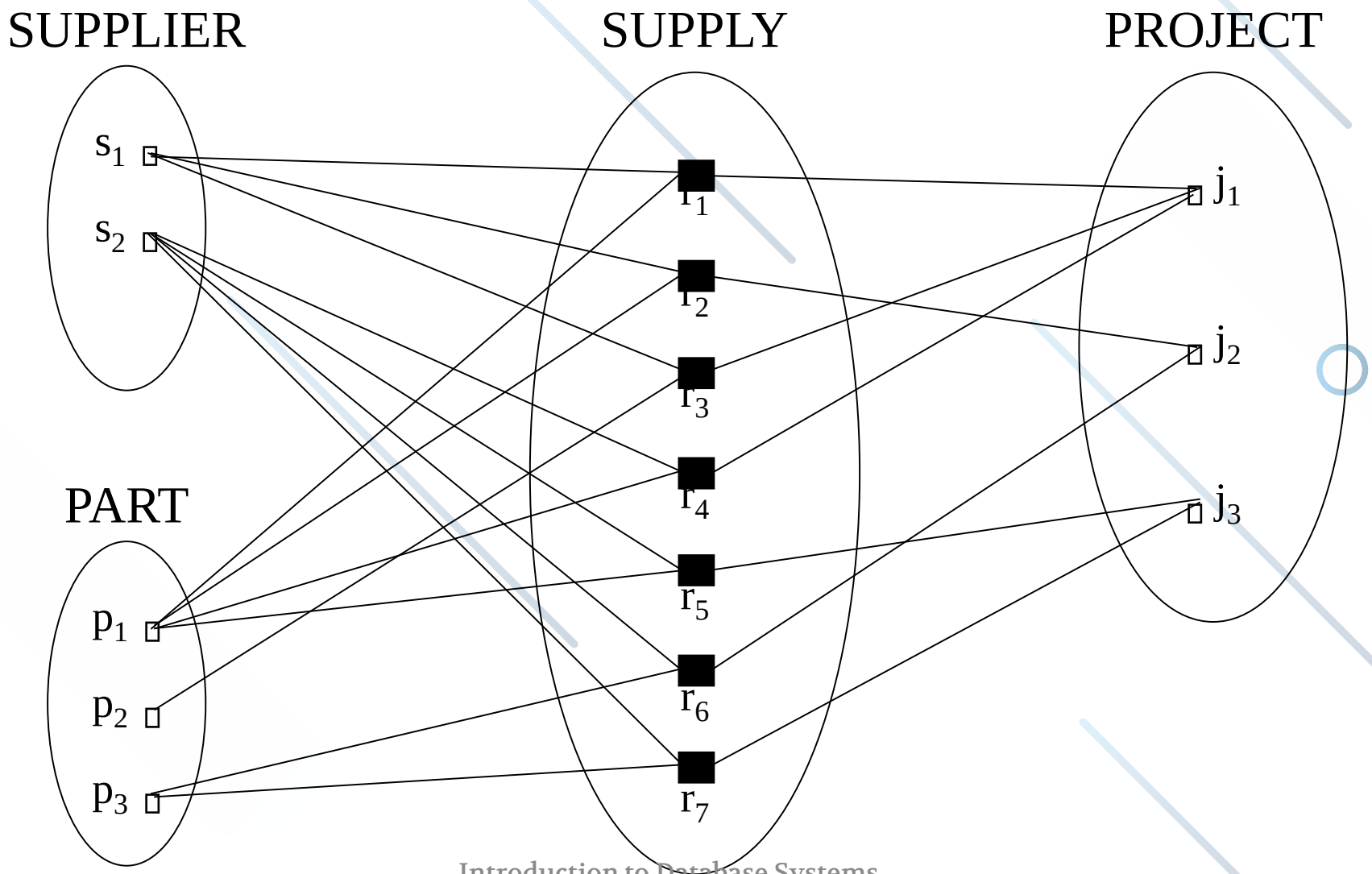
n-ary relationships (n > 2)

- In general, *3 binary relationships can represent different information than a single ternary relationship*
- If needed, the binary and n-ary relationships can all be included in the schema design (where all relationships convey different meanings)
- In some cases, a ternary relationship can be represented as a weak entity if the data model allows a weak entity type to have multiple identifying relationships (and hence multiple owner entity types)

Example of a ternary relationship



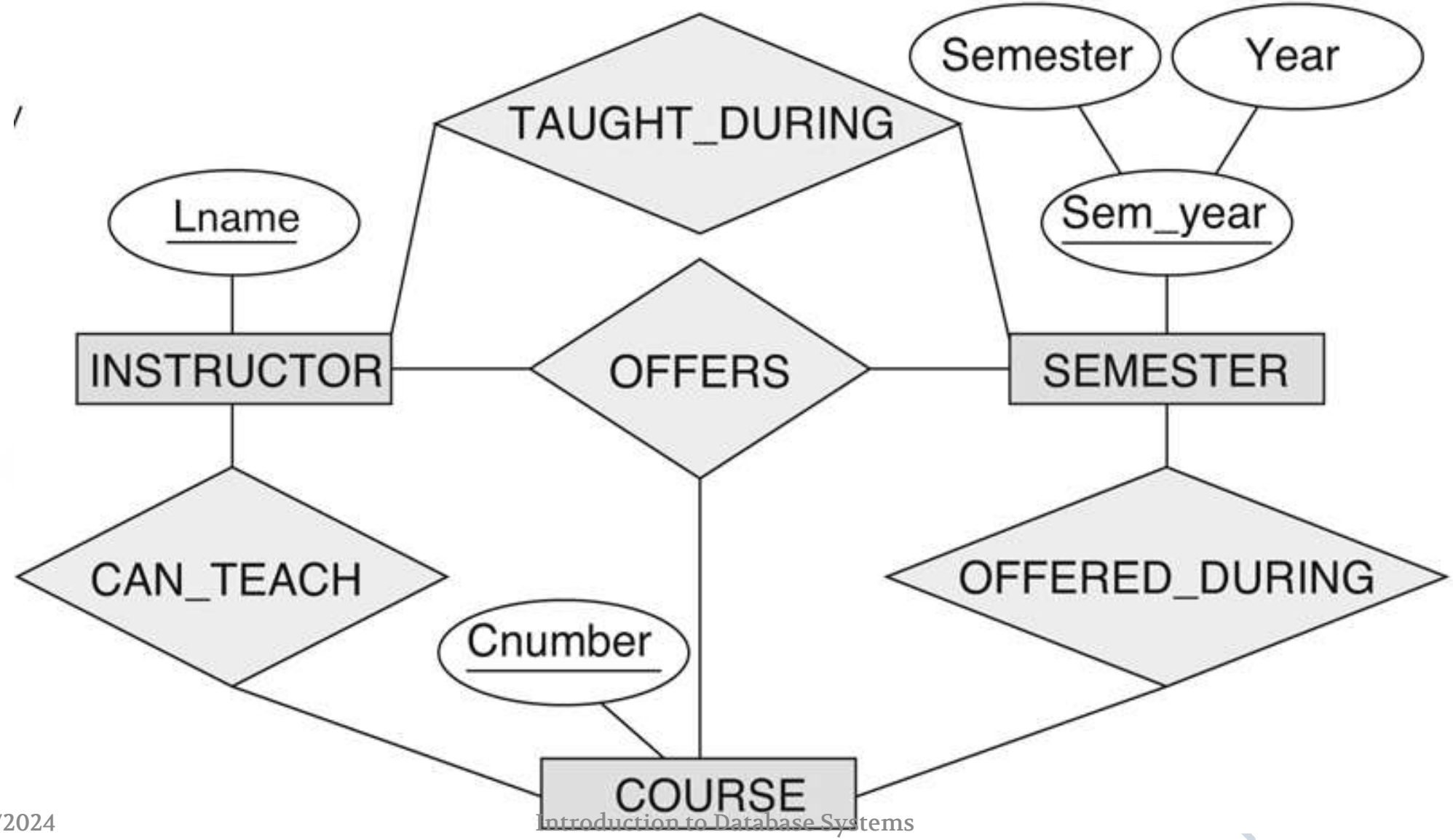
Ternary Relationship Instance Diagram



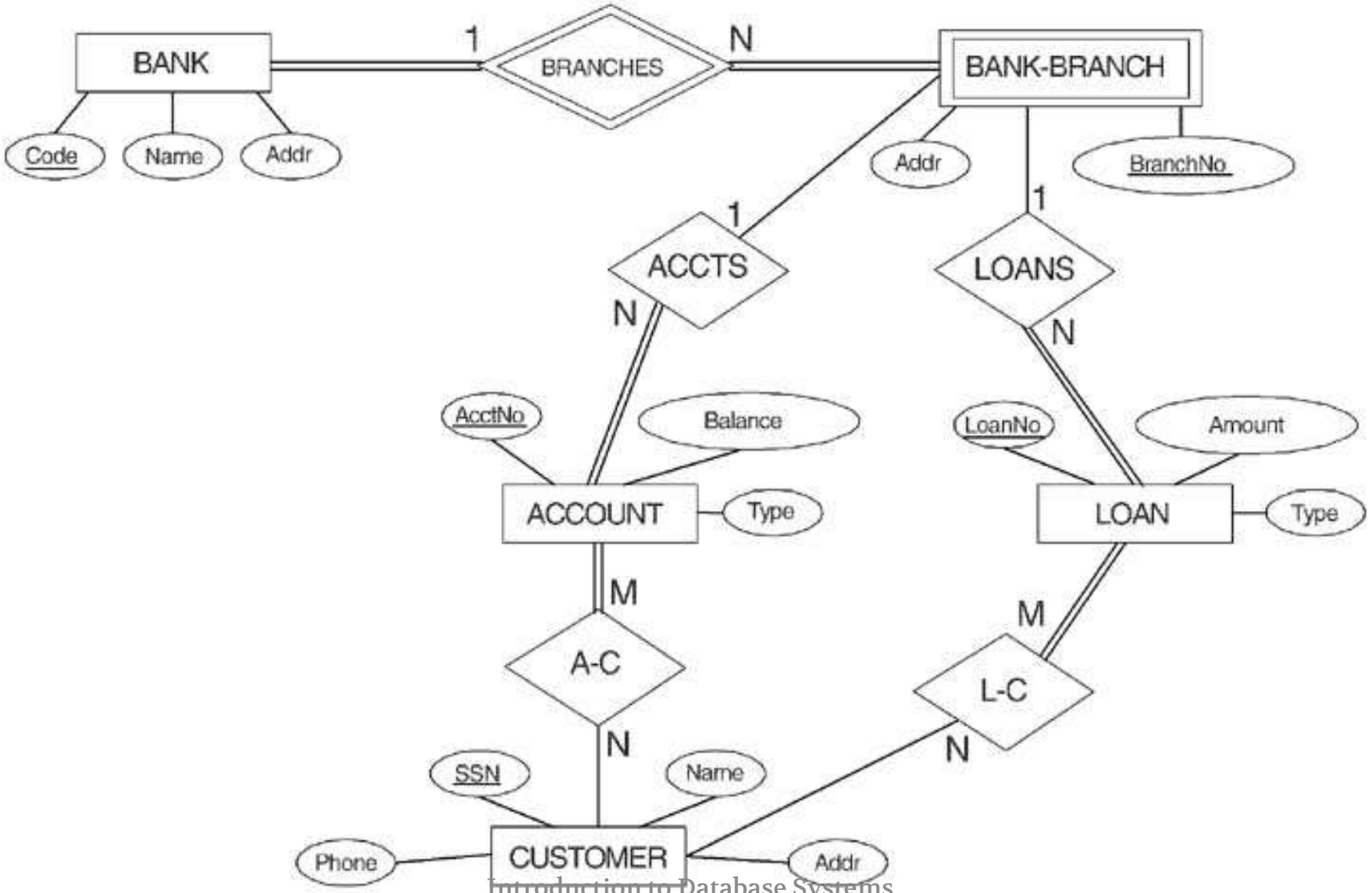
n-ary relationships (n > 2)

- If a particular binary relationship can be derived from a higher-degree relationship at all times, then it is redundant
- For example, the TAUGHT_DURING binary relationship in (next slide) can be derived from the ternary relationship OFFERS (based on the meaning of the relationships)

Another example of a ternary relationship



ER Diagram for A Bank DB



Summary

- Overview of Database Design Process
- Example Database Application (COMPANY)
- ER Model Concepts
- ER Diagrams - Notation
- ER Diagram for COMPANY Schema
- Relationships of Higher Degree

Thank you

Questions?

